UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS

Trabajo de investigación

Reconocimiento automático de frutas y hortalizas mediante redes neuronales convolucionales: comparación de tres arquitecturas sobre un conjunto de 36 clases

Curso: IS056B – Ingeniería del Conocimiento
Docente: Msc. Jaime Antonio Huaytalla Pariona
Semestre: 2026-II

Integrantes

1. Acevedo Pomayay Jean Pool Thair
2. Rouviros Santacrocce Eduardo
3. Alfaro Victoria Gino Paoli
4. Loyola Rodirguez Renzo Styp

Huancayo – Perú
Noviembre de 2026

Índice general

Resumen **v**

Abstract **vi**

1. Introducción **1**
1.1. Planteamiento del problema . . . 1
1.2. Objetivos . . . 1
1.3. Justificación . . . 2
1.4. Alcance y delimitación . . . 2

2. Fundamentos teóricos **2**
2.1. Redes neuronales convolucionales . . . 2
2.2. Normalización por lotes, *dropout* y regularización L2 . . . 3
2.3. Conexiones residuales . . . 3
2.4. Aumento de datos . . . 3
2.5. Optimización . . . 3
2.6. Métricas de evaluación . . . 4

3. Metodología **4**
3.1. Fase 1: definición del problema . . . 4
3.2. Fase 2: conjunto de datos y análisis exploratorio . . . 5
3.3. Fase 3: preprocesamiento y aumento de datos . . . 5
3.4. Fase 4: arquitecturas . . . 5
3.5. Fase 5: configuración de entrenamiento . . . 6

4. Resultados **7**
4.1. Fase 2: análisis exploratorio de datos . . . 7
4.1.1. Composición del conjunto de datos . . . 7
4.1.2. Dimensiones, canales e integridad . . . 9
4.1.3. Inspección visual . . . 10
4.2. Fase 3: pipelines de datos . . . 10
4.3. Fase 4: parámetros de los modelos . . . 10
4.4. Fase 5: entrenamiento y curvas de aprendizaje . . . 11
4.4.1. Registro del entrenamiento . . . 11
4.4.2. Curvas de aprendizaje . . . 11
4.4.3. Análisis comparativo de las curvas . . . 13
4.5. Fase 6: evaluación completa en el conjunto de prueba . . . 13
4.5.1. Selección del modelo . . . 13
4.5.2. Métricas globales . . . 14
4.5.3. Informe de clasificación por clase . . . 14
4.5.4. Matriz de confusión . . . 17
4.5.5. Curvas ROC multiclase . . . 17
4.6. Fase 7: análisis de errores . . . 19
4.6.1. Pares de clases más confundidos . . . 19
4.6.2. Interpretación de las confusiones . . . 19
4.6.3. Inspección visual de errores . . . 20

4.6.4. Acciones correctivas 21
4.7. Fase 8: comparación sistemática de modelos 22
4.8. Fases 9, 10 y 11: versionado, *logging* y documentación 22
4.9. Fase 12: despliegue y conversión a TensorFlow Lite 23

5. Discusión **24**

6. Conclusiones **25**
6.1. Limitaciones 25
6.2. Trabajo futuro 25

Referencias **27**

A. Código principal del proyecto **28**
A.1. Pipeline de datos (Fase 3) 28
A.2. CNN Baseline (Fase 4) 28
A.3. Bloque residual (Fase 4) 29
A.4. Entrenamiento con cinco callbacks (Fase 5) 29
A.5. Conversión a TensorFlow Lite (Fase 12) 29

Índice de figuras

1. Distribución total de imágenes por clase. Salida del bloque de código de la Fase 2. 8
2. Muestras del conjunto de entrenamiento (dos imágenes por clase para las seis primeras clases). 10
3. Curvas de *accuracy* y pérdida de la CNN Baseline (entrenamiento y validación). 11
4. Curvas de *accuracy* y pérdida de la CNN Mejorada. 12
5. Curvas de *accuracy* y pérdida de la CNN Optimizada. 12
6. Validación por época de los tres modelos (elaboración propia a partir de los registros de entrenamiento). 13
7. F1-score por clase (elaboración propia a partir del informe de clasificación). Rojo: F1 < 0.40; azul: F1 ≥ 0.75; gris: intermedio. 16
9. Curvas ROC multiclase (uno contra el resto) de las 36 clases y AUC por clase. 17
8. Matriz de confusión de la CNN Baseline en el conjunto de prueba (filas: clase real; columnas: clase predicha). 18
10. Ocho primeras predicciones incorrectas del conjunto de prueba, con su etiqueta real y la predicha. 20
11. Izquierda: *accuracy* de validación por modelo. Derecha: tamaño estimado en FP32 frente al límite de 10 MB. 22

Índice de tablas

1. Entorno de ejecución y parámetros globales (Fase 0). 4
2. Definición del problema (Fase 1). 5
3. Comparación estructural de las arquitecturas (Fase 4). 6
4. Hiperparámetros de entrenamiento, idénticos para los tres modelos. . . 6
5. Número de imágenes por clase y subconjunto (resultado de la Fase 2). . 7
6. Dimensiones y canales de una muestra de 360 imágenes de entrenamiento. 9
7. Métricas de la última época (época 30) de cada modelo. La pérdida de entrenamiento de los modelos con L2 incluye el término de penalización. 11
8. Métricas globales de la CNN Baseline en el conjunto de prueba ($n = 359$). 14
9. Informe de clasificación por clase en el conjunto de prueba (CNN Baseline). 14
10. Diez confusiones más frecuentes en el conjunto de prueba. 19
11. Hallazgos observados y acciones correctivas propuestas. 21
12. Comparación sistemática de los tres modelos. 22
13. Metadatos del modelo en producción (`metadata.json`). 23
14. Tamaño real de los archivos TensorFlow Lite. 23

Resumen

Este trabajo documenta el desarrollo completo de un sistema de clasificación multiclase de imágenes de frutas y hortalizas basado en redes neuronales convolucionales (CNN). Se utilizó un conjunto de 3 825 imágenes distribuidas en 36 clases y divididas en subconjuntos de entrenamiento (3 115), validación (351) y prueba (359). Se diseñaron y entrenaron tres arquitecturas desde cero: una CNN base de 115 076 parámetros, una CNN con conexiones residuales de 329 860 parámetros y una variante optimizada con regularización L2, *dropout* adicional y aumento de datos incorporado al modelo (335 812 parámetros). Todas se entrenaron durante 30 épocas con el optimizador AdamW y cinco *callbacks* de control (`ModelCheckpoint`, `EarlyStopping`, `ReduceLROnPlateau`, `TensorBoard` y `CSVLogger`).

El modelo seleccionado por su desempeño en validación fue la CNN base, con una *accuracy* de validación de 0.6268. En el conjunto de prueba, que no participó en el entrenamiento ni en la selección, alcanzó una *accuracy* de 0.6240, una *accuracy* Top-3 de 0.8607, un Macro F1-score de 0.6025 y un Macro AUC (uno contra el resto) de 0.9774. Este resultado equivale a aproximadamente 20 veces el desempeño del clasificador mayoritario (3.14 %) y a 22 veces el del azar (2.78 %). El análisis de errores mostró que las confusiones más frecuentes ocurren entre categorías de nombre y apariencia casi equivalentes (*corn*/*sweetcorn*, *bell pepper*/*capsicum*/*paprika*) y que el conjunto contiene imágenes no fotográficas (logotipos, caricaturas, carteles). Las curvas de aprendizaje indican subajuste y no sobreajuste: ninguna arquitectura había convergido al completar las 30 épocas. El modelo se exportó a TensorFlow Lite con un tamaño de 0.44 MB (0.22 MB con cuantización a float16), muy por debajo del límite de 10 MB planteado para uso móvil.

Palabras clave: redes neuronales convolucionales, clasificación de imágenes, frutas y hortalizas, aprendizaje profundo, TensorFlow Lite, análisis de errores.

Abstract

This work documents the end-to-end development of a multiclass image classification system for fruits and vegetables based on convolutional neural networks (CNNs). A dataset of 3,825 images spanning 36 classes was used, split into training (3,115), validation (351) and test (359) subsets. Three architectures were designed and trained from scratch: a baseline CNN with 115,076 parameters, a residual CNN with 329,860 parameters, and an optimized variant with L2 regularization, extra dropout and in-model data augmentation (335,812 parameters). All models were trained for 30 epochs with the AdamW optimizer and five control callbacks (`ModelCheckpoint`, `EarlyStopping`, `ReduceLROnPlateau`, `TensorBoard` and `CSVLogger`).

The model selected on validation performance was the baseline CNN, with a validation accuracy of 0.6268. On the held-out test set, it reached an accuracy of 0.6240, a Top-3 accuracy of 0.8607, a Macro F1-score of 0.6025 and a one-vs-rest Macro AUC of 0.9774, roughly 20 times the majority-class baseline (3.14%) and 22 times chance (2.78%). Error analysis showed that the most frequent confusions involve categories with nearly equivalent names and appearance (corn/sweetcorn, bell pepper/capsicum/paprika), and that the dataset contains non-photographic images (logos, cartoons, posters). Learning curves indicate underfitting rather than overfitting: none of the architectures had converged after 30 epochs. The model was exported to TensorFlow Lite at 0.44 MB (0.22 MB with float16 quantization), far below the 10 MB mobile-deployment limit.

Keywords: convolutional neural networks, image classification, fruits and vegetables, deep learning, TensorFlow Lite, error analysis.

1. Introducción

La identificación automática de alimentos a partir de imágenes tiene aplicaciones directas en el comercio minorista, la logística agrícola, la educación nutricional y la accesibilidad. En particular, distinguir frutas y hortalizas es un problema visualmente exigente: muchas especies comparten forma, color y textura, las fotografías se toman en condiciones de iluminación y fondo muy distintas, y una misma especie recibe nombres diferentes según la región. Las redes neuronales convolucionales (CNN) son hoy el enfoque estándar para este tipo de tarea, porque aprenden jerarquías de características visuales directamente de los píxeles [9, 3].

Este trabajo presenta el desarrollo íntegro de un clasificador multiclase de imágenes de frutas y hortalizas, siguiendo un flujo de trabajo reproducible de doce fases: definición del problema, análisis exploratorio, preprocesamiento, diseño de modelos, entrenamiento, evaluación, análisis de errores, comparación sistemática, versionado, *logging*, documentación y despliegue. Cada fase se documenta con los resultados reales obtenidos al ejecutar el *notebook* del proyecto, y cada resultado se interpreta con sus limitaciones.

1.1. Planteamiento del problema

Dado un conjunto de imágenes etiquetadas con 36 categorías de frutas y hortalizas, se requiere un modelo que asigne a una imagen nueva la categoría correcta. Se trata de un problema de clasificación multiclase supervisada. La pregunta central es: *¿qué nivel de desempeño puede alcanzar una CNN entrenada desde cero, con recursos computacionales modestos, y qué tipos de error persisten?*

1.2. Objetivos

Objetivo general. Desarrollar y evaluar un sistema de clasificación de imágenes de frutas y hortalizas mediante CNN, y determinar su viabilidad para despliegue en dispositivos móviles.

Objetivos específicos.

1. Caracterizar el conjunto de datos mediante análisis exploratorio (distribución por clase, dimensiones, canales e integridad de archivos).
2. Diseñar, entrenar y comparar tres arquitecturas CNN con una misma configuración de entrenamiento.
3. Evaluar el mejor modelo sobre un conjunto de prueba independiente con métricas

globales y por clase.

4. Analizar los errores de clasificación y proponer acciones correctivas basadas en evidencia.
5. Exportar el modelo a TensorFlow Lite y verificar el cumplimiento del límite de tamaño de 10 MB.

1.3. Justificación

Justificación práctica. Un clasificador ligero de frutas y hortalizas puede integrarse en aplicaciones móviles para balanzas de autoservicio, inventarios o herramientas educativas. Para ello el modelo debe ser pequeño y rápido; por eso este trabajo evalúa explícitamente el tamaño en disco y la conversión a TensorFlow Lite.

Justificación metodológica. Comparar arquitecturas bajo condiciones idénticas (misma semilla, mismos datos, mismos hiperparámetros y mismos *callbacks*) permite atribuir las diferencias de rendimiento a la arquitectura y no al procedimiento. Además, reportar el análisis de errores y las limitaciones del conjunto de datos aporta evidencia sobre qué mejorar primero: el modelo o los datos.

Justificación académica. El trabajo integra, en un solo caso, los contenidos de Ingeniería del Conocimiento: representación del problema, aprendizaje supervisado, evaluación con métricas apropiadas, interpretación de resultados y puesta en producción.

1.4. Alcance y delimitación

El estudio se limita a las 36 clases del conjunto de datos disponible y a modelos entrenados desde cero (sin aprendizaje por transferencia). El desempeño se mide sobre el subconjunto de prueba del propio conjunto de datos; no se evaluó con fotografías tomadas fuera de él.

2. Fundamentos teóricos

2.1. Redes neuronales convolucionales

Una CNN procesa una imagen mediante capas convolucionales que aplican filtros locales y comparten sus pesos en toda la imagen, lo que reduce el número de parámetros y aporta equivariancia a traslaciones [9]. Entre convoluciones se intercalan funciones de activación no lineales (aquí ReLU), capas de *pooling* que reducen la resolución espacial

y, al final, capas densas que producen la probabilidad de cada clase mediante la función *softmax*:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K, \tag{1}$$

donde z_k es la puntuación (*logit*) de la clase k y $K = 36$. La función de pérdida empleada es la entropía cruzada categórica dispersa, $\mathcal{L} = -\log \hat{p}_y$, donde y es la clase verdadera.

2.2. Normalización por lotes, *dropout* y regularización L2

La normalización por lotes (*batch normalization*) estandariza las activaciones de cada capa y estabiliza el entrenamiento [6]. *Dropout* desactiva aleatoriamente neuronas durante el entrenamiento para reducir la co-adaptación [13]. La regularización L2 penaliza la magnitud de los pesos añadiendo $\lambda \sum w^2$ a la pérdida; en este trabajo $\lambda = 10^{-4}$. Debe notarse que esta penalización se suma a la pérdida de entrenamiento reportada por Keras, por lo que la pérdida de entrenamiento de modelos con L2 no es directamente comparable con la de modelos sin L2.

2.3. Conexiones residuales

Un bloque residual calcula $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$, de modo que la red aprende una corrección sobre la identidad. Esto facilita la propagación del gradiente en redes profundas [4]. Cuando cambia la resolución o el número de filtros, el atajo se proyecta con una convolución 1×1.

2.4. Aumento de datos

El aumento de datos genera variaciones plausibles de las imágenes de entrenamiento (volteos, rotaciones, zoom) para mejorar la generalización [11]. Se aplica solo durante el entrenamiento; validación y prueba usan las imágenes originales.

2.5. Optimización

Se utilizó AdamW, que desacopla la disminución de pesos (*weight decay*) de la actualización adaptativa de Adam [8, 10].

2.6. Métricas de evaluación

Sean TP_k, FP_k y FN_k los verdaderos positivos, falsos positivos y falsos negativos de la clase k:

$$P_k = \frac{TP_k}{TP_k + FP_k}, \qquad R_k = \frac{TP_k}{TP_k + FN_k}, \qquad F1_k = \frac{2P_kR_k}{P_k + R_k}. \tag{2}$$

El **Macro F1** es el promedio simple de $F1_k$ sobre las K clases, y trata a todas por igual [12]. La **accuracy Top-3** cuenta un acierto cuando la clase verdadera está entre las tres con mayor probabilidad. La curva ROC grafica la tasa de verdaderos positivos frente a la de falsos positivos al variar el umbral; el **AUC** es el área bajo ella [2]. En el caso multiclase se emplea el esquema *uno contra el resto* (OvR) y se promedia el AUC de las clases (Macro AUC).

3. Metodología

El trabajo se desarrolló en Python con TensorFlow/Keras [1] sobre Google Colab. La Tabla 1 resume el entorno y los parámetros globales (Fase 0).

Tabla 1: Entorno de ejecución y parámetros globales (Fase 0).

Elemento	**Valor**
TensorFlow	2.21.0
Python	3.13.16
Aceleración	1 GPU detectada (`GPU:0`)
Semilla aleatoria	42 (`random`, `numpy`, `tensorflow`)
Tamaño de imagen	224×224 píxeles, 3 canales
Tamaño de lote	32
Épocas máximas	30

3.1. Fase 1: definición del problema

La Tabla 2 sintetiza la definición formal del problema. Se adoptó Macro F1 como métrica principal junto con la *accuracy*, porque el primero pondera por igual todas las clases aunque estén desbalanceadas. Se definieron dos líneas base: el clasificador mayoritario, que siempre predice la clase más frecuente, y el azar uniforme (1/36).

Tabla 2: Definición del problema (Fase 1).

Aspecto	**Definición**
Problema	Clasificar imágenes de frutas y hortalizas en su categoría para apoyar su identificación automática.
Tipo de problema	Clasificación multiclase supervisada (36 clases).
Métricas	Macro F1-score (principal), *accuracy*, Top-3 y Macro AUC.
Restricciones	Calidad y cantidad de imágenes, similitud visual entre categorías, capacidad de cómputo y tamaño del modelo para despliegue.
Línea base	Mayoritario: proporción de la clase más frecuente. Secundaria: azar, 1/36.

3.2. Fase 2: conjunto de datos y análisis exploratorio

El conjunto de datos se organizó en tres carpetas (`train`, `validation`, `test`), cada una con una subcarpeta por clase. El código detecta automáticamente la carpeta raíz y verifica que las tres particiones contengan las mismas clases en el mismo orden, condición indispensable para que las etiquetas enteras sean consistentes.

3.3. Fase 3: preprocesamiento y aumento de datos

Las imágenes se redimensionan a 224×224 y se normalizan de $[0, 255]$ a $[0, 1]$ mediante una capa `Rescaling`. El aumento de datos (volteo horizontal, rotación de hasta $\pm 8\,\%$ de vuelta, es decir $\approx \pm 28.8°$, y zoom de $\pm 10\,\%$) se aplica únicamente al subconjunto de entrenamiento. Se usó `prefetch` en los tres subconjuntos y `cache` en validación y prueba para acelerar la lectura; el entrenamiento no se almacena en caché para que el aumento produzca variaciones nuevas en cada época.

3.4. Fase 4: arquitecturas

Se diseñaron tres modelos que reciben tensores de $224 \times 224 \times 3$ y producen 36 probabilidades.

- **CNN Baseline.** Tres bloques `Conv(3×3)-BatchNorm-ReLU-MaxPool` con 32, 64 y 128 filtros, seguidos de *Global Average Pooling*, una capa densa de 128 neuronas, *dropout* 0.4 y capa de salida *softmax*.
- **CNN Mejorada.** Una convolución inicial de 32 filtros y tres bloques residuales

(32, 64 y 128 filtros; los dos últimos con *stride* 2), *Global Average Pooling*, densa de 128 con L2 (10^{-4}) y *dropout* 0.5.

- **CNN Optimizada.** La arquitectura mejorada más tres capas de aumento de datos dentro del modelo, L2 en la convolución inicial, una segunda capa densa de 64 neuronas con L2 y *dropout* 0.3.

Tabla 3: Comparación estructural de las arquitecturas (Fase 4).

Característica	**Baseline**	**Mejorada**	**Optimizada**
Bloques de convolución	3 estándar	1 + 3 residuales	1 + 3 residuales
Filtros por bloque	32/64/128	32/32/64/128	32/32/64/128
Conexiones residuales	No	Sí	Sí
Capas densas ocultas	128	128	128 y 64
Dropout	0.4	0.5	0.5 y 0.3
Regularización L2 (10^{-4})	No	Densa	Conv. inicial y densas
Aumento de datos en el modelo	No	No	Sí
Parámetros	**115 076**	**329 860**	**335 812**
Tamaño FP32 estimado (MB)	**0.44**	**1.26**	**1.28**

El tamaño estimado en FP32 se calcula como número de parámetros $\times 4$ bytes. Cabe señalar un detalle de diseño: la CNN Optimizada incorpora capas de aumento *dentro* del modelo, mientras que el *pipeline* de datos de entrenamiento ya aplica un aumento equivalente (Fase 3). Por tanto, esa variante recibe efectivamente un aumento doble, aspecto que se retoma en la discusión.

3.5. Fase 5: configuración de entrenamiento

Tabla 4: Hiperparámetros de entrenamiento, idénticos para los tres modelos.

Parámetro	**Valor**
Optimizador	AdamW
Tasa de aprendizaje inicial	1×10^{-3}
Weight decay	1×10^{-4}
Tamaño de lote	32 (98 pasos por época)
Épocas máximas	30
Función de pérdida	Entropía cruzada categórica dispersa
Métricas de seguimiento	*accuracy* y Top-3

Se emplearon cinco *callbacks*: `ModelCheckpoint` (guarda el modelo con mayor `val_accuracy`), `EarlyStopping` (monitorea `val_loss`, paciencia 7, restaura los mejores pesos), `ReduceLROnPlateau` (factor 0.5, paciencia 3, mínimo 10^{-6}), `TensorBoard` y `CSVLogger`. Cada modelo se entrenó una vez, en alrededor de 40 minutos según las marcas de tiempo de los directorios de experimento.

Nota de transparencia: el diccionario de configuración del *notebook* registra un parámetro `label_smoothing = 0.05`, pero la función de pérdida se instanció sin ese argumento; por tanto, el suavizado de etiquetas **no** se aplicó en los experimentos reportados.

4. Resultados

4.1. Fase 2: análisis exploratorio de datos

4.1.1. Composición del conjunto de datos

La carga de los tres subconjuntos reportó 3 115 archivos de entrenamiento, 351 de validación y 359 de prueba, todos pertenecientes a 36 clases (Tabla 5). El total es de 3 825 imágenes, con una partición aproximada de 81.4 % / 9.2 % / 9.4 %. La clase más numerosa es *peas* (120 imágenes) y la menos numerosa es *apple* (88); la razón entre ambas es $120/88 \approx 1.36$, de modo que el desbalance es leve. En entrenamiento, el rango por clase va de 68 (*apple*, *ginger*) a 100 imágenes (*grapes*, *peas*).

Tabla 5: Número de imágenes por clase y subconjunto (resultado de la Fase 2).

Clase	Entren.	Valid.	Prueba	Total
apple	68	10	10	88
banana	75	9	9	93
beetroot	88	10	10	108
bell pepper	90	9	10	109
cabbage	92	10	10	112
capsicum	89	10	10	109
carrot	82	9	10	101
cauliflower	79	10	10	99
chilli pepper	87	9	10	106
corn	87	10	10	107
cucumber	94	10	10	114
eggplant	84	10	10	104
garlic	92	10	10	112

continúa en la página siguiente

Clase	Entren.	Valid.	Prueba	Total
ginger	68	10	10	88
grapes	100	9	10	119
jalepeno	88	9	10	107
kiwi	88	10	10	108
lemon	82	10	10	102
lettuce	97	9	10	116
mango	86	10	10	106
onion	94	10	10	114
orange	69	9	10	88
paprika	83	10	10	103
pear	89	10	10	109
peas	100	10	10	120
pineapple	99	10	10	119
pomegranate	79	10	10	99
potato	77	10	10	97
raddish	81	9	10	100
soy beans	97	10	10	117
spinach	97	10	10	117
sweetcorn	91	10	10	111
sweetpotato	69	10	10	89
tomato	92	10	10	112
turnip	98	10	10	118
watermelon	84	10	10	104
Total	**3115**	**351**	**359**	**3825**

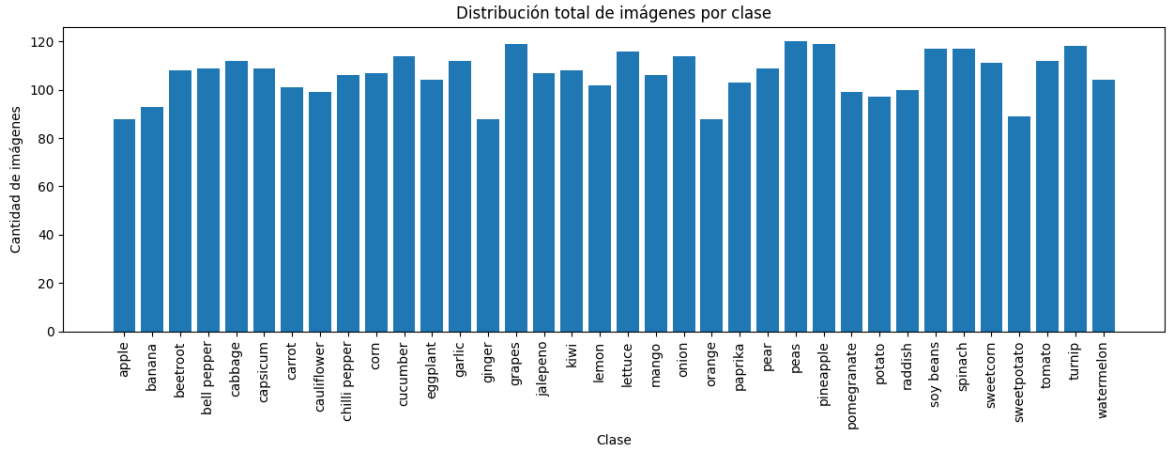


Figura 1: Distribución total de imágenes por clase. Salida del bloque de código de la Fase 2.

La Figura 1 confirma que todas las clases tienen entre 88 y 120 imágenes en total, sin

categorías dominantes ni residuales. Esto justifica el uso de *accuracy* como métrica complementaria, aunque se mantiene Macro F1 como métrica principal. Dado que la clase mayoritaria representa el 3.14 % del total (120/3825) y el azar uniforme el 2.78 % (1/36), ambas líneas base son muy bajas, y cualquier modelo con aprendizaje real las superará con holgura.

4.1.2. Dimensiones, canales e integridad

Se inspeccionó una muestra de 360 imágenes (10 por clase) del subconjunto de entrenamiento. Las dimensiones son altamente heterogéneas (Tabla 6): el ancho varía de 208 a 5 887 píxeles y el alto de 208 a 5 344, con promedios de 1 156.3 y 975.6 píxeles. El 94.7 % de la muestra tiene tres canales (RGB), el 5.0 % tiene cuatro (RGBA, con transparencia) y una imagen es de un solo canal (escala de grises). Esta heterogeneidad obliga a redimensionar todas las imágenes a 224×224 y a convertirlas a RGB, lo cual hace la carga de Keras por defecto. Una consecuencia es que imágenes no cuadradas se deforman al ajustarse al tamaño fijo.

Tabla 6: Dimensiones y canales de una muestra de 360 imágenes de entrenamiento.

Atributo	**Mínimo**	**Máximo**	**Promedio**
Ancho (px)	208	5 887	1 156.3
Alto (px)	208	5 344	975.6
Canales	3 canales: 341; 4 canales: 18; 1 canal: 1		

La verificación de integridad con `Image.verify()` revisó las 3 825 imágenes y no halló ninguna corrupta (0.00 % en entrenamiento, validación y prueba).

4.1.3. Inspección visual



Figura 2: Muestras del conjunto de entrenamiento (dos imágenes por clase para las seis primeras clases).

La Figura 2 revela varias propiedades relevantes. Primero, hay gran variabilidad de fondo, escala y encuadre: desde frutas sobre fondo blanco (*apple*, *beetroot*) hasta escenas de huerto o puestos de mercado con múltiples ejemplares (*bell pepper*, *capsicum*). Segundo, existen imágenes atípicas: una manzana con un rostro superpuesto, una silueta verde sobre fondo negro etiquetada como *apple* y una fotografía con el texto "Nutrition Facts". Tercero, algunas imágenes contienen marcas de agua. Estas observaciones indican que el conjunto fue recopilado de la web con cierto ruido, hecho que condiciona el techo de desempeño alcanzable y que reaparece en el análisis de errores.

4.2. Fase 3: pipelines de datos

La ejecución de la Fase 3 confirmó la construcción de los tres *pipelines* ("Pipelines listos"). No genera gráficas. Sus efectos se aprecian en las curvas de la Fase 5: el aumento se aplica solo al entrenamiento, por lo que las métricas de entrenamiento se calculan sobre imágenes perturbadas y las de validación sobre imágenes originales.

4.3. Fase 4: parámetros de los modelos

La instanciación de los modelos reportó 115 076 parámetros para la CNN Baseline (0.44 MB en FP32), 329 860 para la Mejorada (1.26 MB) y 335 812 para la Optimizada (1.28 MB). La arquitectura Mejorada tiene 2.87 veces los parámetros de la base, y la Optimizada solo 5 952 más que la Mejorada, ya que las capas de aumento no tienen parámetros entrenables y la capa densa de 64 neuronas añade una cantidad reducida. Los tres modelos son pequeños comparados con redes de referencia, lo que favorece el despliegue móvil.

4.4. Fase 5: entrenamiento y curvas de aprendizaje

4.4.1. Registro del entrenamiento

El entrenamiento de los tres modelos recorrió las 30 épocas: `EarlyStopping` no se activó en ningún caso porque `val_loss` seguía mejorando de forma intermitente. `ReduceLROnPlateau` redujo la tasa de aprendizaje cuatro veces en la CNN Baseline (épocas 11, 14, 21 y 28; de 10^{-3} a 6.25×10^{-5}) y tres veces en la Mejorada (épocas 10, 14 y 27) y en la Optimizada (épocas 12, 22 y 28). La Tabla 7 resume el estado de cada modelo al final.

Tabla 7: Métricas de la última época (época 30) de cada modelo. La pérdida de entrenamiento de los modelos con L2 incluye el término de penalización.

Modelo	Acc. entren.	Acc. valid.	Loss entren.	Loss valid.
CNN Baseline	0.4584	0.6182	1.8184	1.3013
CNN Mejorada	0.4180	0.6239	1.9448	1.4142
CNN Optimizada	0.3352	0.5043	2.2314	1.7353

4.4.2. Curvas de aprendizaje

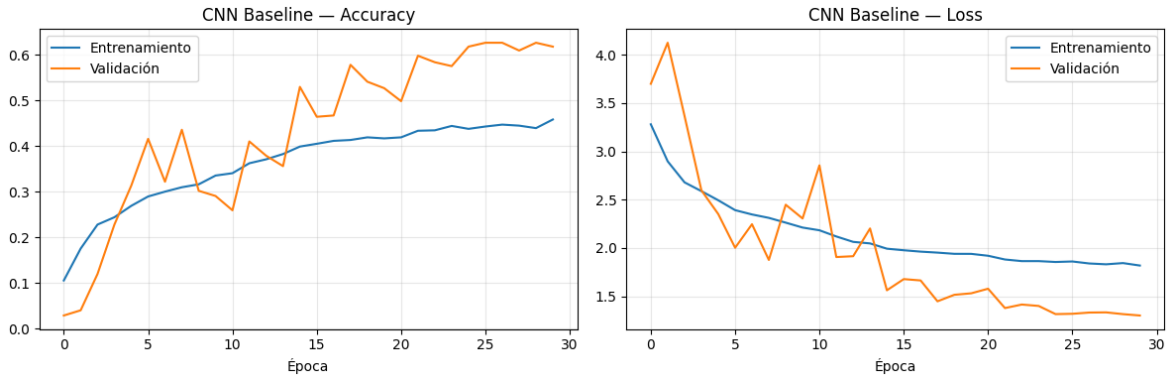


Figura 3: Curvas de *accuracy* y pérdida de la CNN Baseline (entrenamiento y validación).

CNN Baseline (Figura 3). La *accuracy* de entrenamiento crece de forma casi monótona desde 0.105 hasta 0.458, y la pérdida de entrenamiento baja de 3.28 a 1.82. La validación es muy ruidosa durante las primeras 20 épocas (por ejemplo, cae de 0.436 en la época 8 a 0.259 en la época 11) y se estabiliza después de que la tasa de aprendizaje se reduce, alcanzando su máximo de 0.6268 en la época 26 y una pérdida mínima de 1.3013 en la época 30. Destaca que la validación supera a la de entrenamiento por unos 16 puntos porcentuales; esto **no** es una anomalía de fuga de datos, sino el efecto combinado de (i) el aumento de datos y el *dropout*, que solo penalizan el entrenamiento,

(ii) que la métrica de entrenamiento es un promedio sobre la época mientras los pesos cambian, y (iii) el tamaño pequeño del conjunto de validación (351 imágenes), que hace ruidosa la estimación.

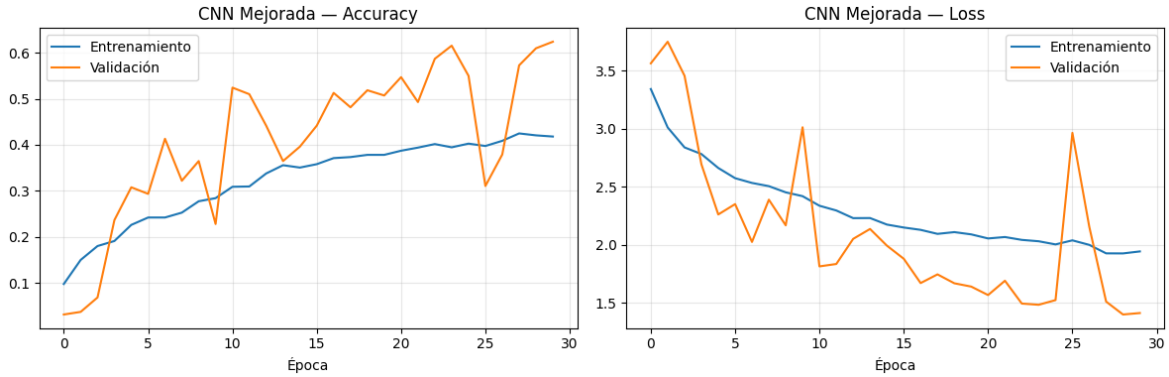


Figura 4: Curvas de *accuracy* y pérdida de la CNN Mejorada.

CNN Mejorada (Figura 4). Presenta el mismo patrón, con mayor inestabilidad intermedia: la validación cae a 0.2279 en la época 10 y a 0.3105 en la época 26, caídas que probablemente se deben a la inestabilidad de las estadísticas de *batch normalization* en las primeras épocas, aunque este trabajo no lo comprobó. Tras los descensos de la tasa de aprendizaje se estabiliza y finaliza con 0.6239 de *accuracy* de validación (su máximo, en la época 30) y una pérdida mínima de 1.4016 (época 29). Su brecha de validación sobre entrenamiento es de 20.6 puntos.

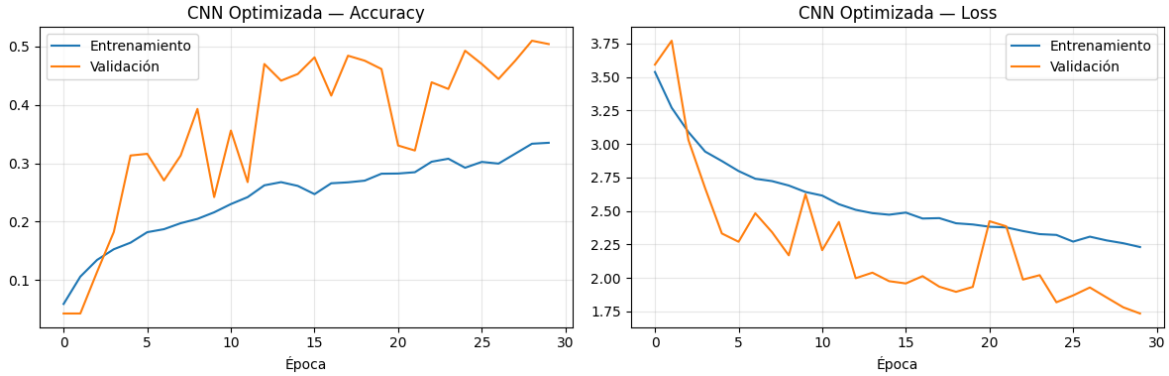


Figura 5: Curvas de *accuracy* y pérdida de la CNN Optimizada.

CNN Optimizada (Figura 5). Es la de aprendizaje más lento: la *accuracy* de entrenamiento llega solo a 0.335 y la de validación a un máximo de 0.5100 (época 29). Las curvas de entrenamiento y validación continúan descendiendo y ascendiendo, respectivamente, al terminar la época 30, es decir, el modelo no había convergido.

4.4.3. Análisis comparativo de las curvas

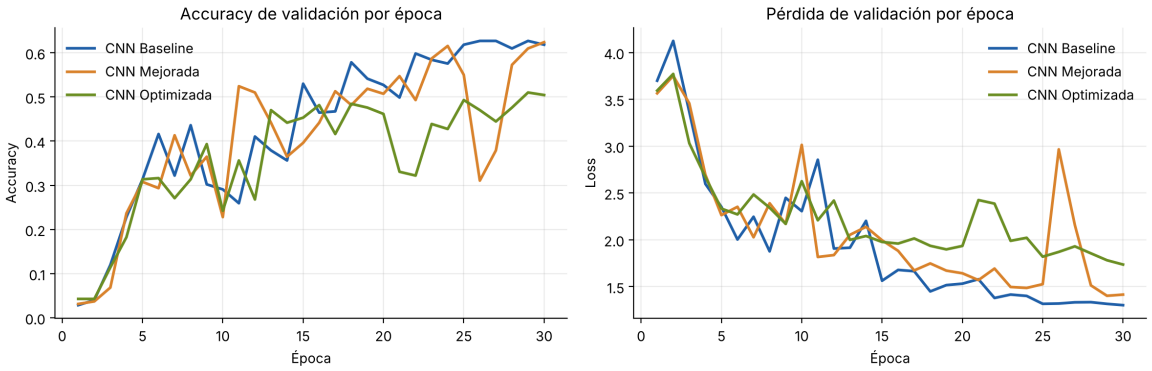


Figura 6: Validación por época de los tres modelos (elaboración propia a partir de los registros de entrenamiento).

La Figura 6 superpone la validación de los tres modelos. Las observaciones principales son:

- **No hay sobreajuste.** El diagnóstico de sobreajuste exige que la precisión de entrenamiento siga subiendo mientras la de validación se estanca o empeora y la pérdida de validación aumenta. Aquí ocurre lo contrario: ambas pérdidas disminuyen hasta el final y la validación supera al entrenamiento. El diagnóstico apropiado es **subajuste**: los modelos aún tienen capacidad de aprender más con más épocas.
- **Baseline y Mejorada son prácticamente equivalentes** en *accuracy* de validación (0.6268 frente a 0.6239); la diferencia de 0.0029 equivale exactamente a una imagen de las 351 de validación.
- **La CNN Optimizada rinde peor** (0.5100) y converge más despacio.

4.5. Fase 6: evaluación completa en el conjunto de prueba

4.5.1. Selección del modelo

El *notebook* seleccionó automáticamente el modelo con mayor *accuracy* de validación, que fue la **CNN Baseline**, y cargó su mejor *checkpoint* (época 26). La selección es defendible por tres razones adicionales: su pérdida de validación mínima es la menor (1.3013 frente a 1.4016 y 1.7353), es el modelo más pequeño (0.44 MB) y su diferencia con la Mejorada en *accuracy* equivale a una sola imagen, es decir, un empate estadístico en el que prevalece la parsimonia.

4.5.2. Métricas globales

Tabla 8: Métricas globales de la CNN Baseline en el conjunto de prueba ($n = 359$).

Métrica	**Valor**
Accuracy	0.6240 (224 de 359 imágenes)
Pérdida (entropía cruzada)	1.3205
Accuracy Top-3	0.8607 (309 de 359 imágenes)
Macro Precisión	0.6575
Macro Recall	0.6241
Macro F1-score	0.6025
Weighted F1-score	0.6021
Macro AUC (OvR)	0.9774

Interpretación. El modelo clasifica correctamente 224 de 359 imágenes (62.4 %), prácticamente igual a su validación (62.7 %), lo que indica que la selección no sobreestimó el rendimiento. Frente a las líneas base, la *accuracy* equivale a $0.6240/0.0314 \approx 19.9$ veces el clasificador mayoritario y a $0.6240/0.0278 \approx 22.5$ veces el azar. La *accuracy* Top-3 de 86.1 % muestra que, aun cuando la primera predicción falla, la clase correcta suele figurar entre las tres más probables: 85 de las 135 imágenes mal clasificadas en primera opción tienen la etiqueta correcta en el segundo o tercer lugar. Macro F1 (0.6025) y Weighted F1 (0.6021) casi coinciden porque el conjunto de prueba está prácticamente balanceado (10 imágenes por clase, salvo *banana* con 9). El Macro F1 es inferior a la *accuracy* porque las clases débiles pesan lo mismo que las fuertes.

4.5.3. Informe de clasificación por clase

La Tabla 9 reproduce el informe de clasificación completo, y la Figura 7 lo resume gráficamente.

Tabla 9: Informe de clasificación por clase en el conjunto de prueba (CNN Baseline).

Clase	**Precisión**	**Recall**	**F1-score**	**Soporte**
apple	1.0000	0.3000	0.4615	10
banana	0.8571	0.6667	0.7500	9
beetroot	0.5000	0.6000	0.5455	10
bell pepper	0.6667	0.2000	0.3077	10
cabbage	0.6667	0.6000	0.6316	10

continúa en la página siguiente

Clase	**Precisión**	**Recall**	**F1-score**	**Soporte**
capsicum	0.3846	0.5000	0.4348	10
carrot	0.7692	1.0000	0.8696	10
cauliflower	0.7500	0.6000	0.6667	10
chilli pepper	0.3636	0.4000	0.3810	10
corn	0.3333	0.1000	0.1538	10
cucumber	0.3333	0.4000	0.3636	10
eggplant	0.7000	0.7000	0.7000	10
garlic	0.7692	1.0000	0.8696	10
ginger	0.5833	0.7000	0.6364	10
grapes	0.6667	0.8000	0.7273	10
jalepeno	1.0000	0.1000	0.1818	10
kiwi	0.6250	1.0000	0.7692	10
lemon	0.9000	0.9000	0.9000	10
lettuce	0.7273	0.8000	0.7619	10
mango	0.6667	0.4000	0.5000	10
onion	0.7143	0.5000	0.5882	10
orange	0.7500	0.6000	0.6667	10
paprika	0.7000	0.7000	0.7000	10
pear	0.8750	0.7000	0.7778	10
peas	0.6250	1.0000	0.7692	10
pineapple	0.5625	0.9000	0.6923	10
pomegranate	0.5294	0.9000	0.6667	10
potato	1.0000	0.5000	0.6667	10
raddish	0.5556	0.5000	0.5263	10
soy beans	0.6000	0.9000	0.7200	10
spinach	0.2857	0.2000	0.2353	10
sweetcorn	0.4167	1.0000	0.5882	10
sweetpotato	0.6000	0.3000	0.4000	10
tomato	0.8182	0.9000	0.8571	10
turnip	0.6250	0.5000	0.5556	10
watermelon	0.7500	0.6000	0.6667	10
Accuracy			0.6240	359
Macro avg	0.6575	0.6241	0.6025	359
Weighted avg	0.6569	0.6240	0.6021	359

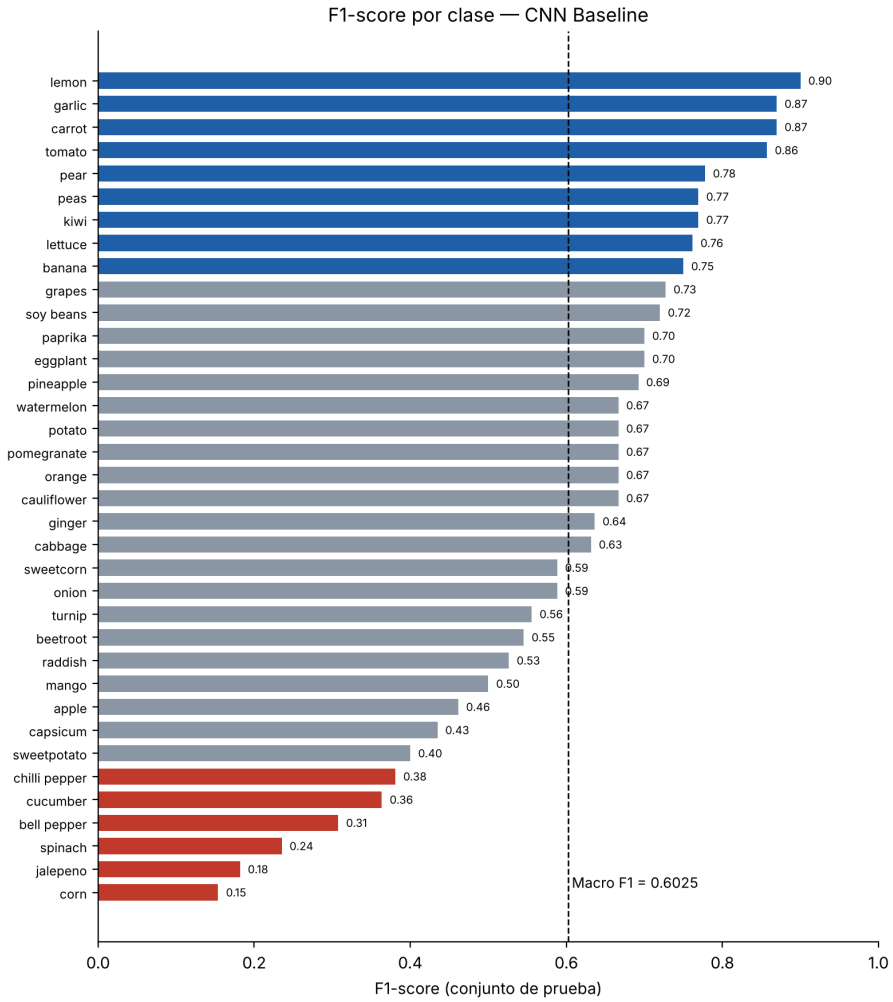


Figura 7: F1-score por clase (elaboración propia a partir del informe de clasificación). Rojo: F1 < 0.40; azul: F1 ≥ 0.75; gris: intermedio.

Interpretación por clase.

- **Mejores clases:** *lemon* (F1 0.9000), *carrot* y *garlic* (0.8696 ambas, con recall de 1.0000), *tomato* (0.8571), *kiwi* (0.7692) y *peas* (0.7692). Son objetos de color o forma muy característicos y poco ambiguos.
- **Peores clases:** *corn* (F1 0.1538; recall 0.1000), *jalepeno* (0.1818; recall 0.1000), *spinach* (0.2353), *bell pepper* (0.3077; recall 0.2000), *cucumber* (0.3636) y *chilli pepper* (0.3810).
- **Precisión alta con recall bajo.** *apple*, *jalepeno* y *potato* tienen precisión 1.0000 pero recall de 0.3, 0.1 y 0.5. El modelo casi nunca se equivoca cuando las predice, pero deja pasar la mayoría de sus imágenes; es un comportamiento conservador.
- **Recall alto con precisión baja.** *sweetcorn* tiene recall 1.0000 y precisión 0.4167: de 24 imágenes que el modelo asignó a *sweetcorn*, solo 10 lo eran. Esta clase actúa como "sumidero" de otras categorías (véase la Fase 7).

4.5.4. Matriz de confusión

La diagonal de la Figura 8 concentra la mayor parte de los aciertos: *carrot*, *garlic*, *kiwi*, *peas* y *sweetcorn* obtienen 10 de 10. Fuera de la diagonal se observan pocas celdas intensas, lo que indica que los errores no están dispersos sino concentrados en pares específicos de clases, como se detalla en la Fase 7. La celda más intensa fuera de la diagonal es *corn* → *sweetcorn* (8 imágenes).

4.5.5. Curvas ROC multiclase

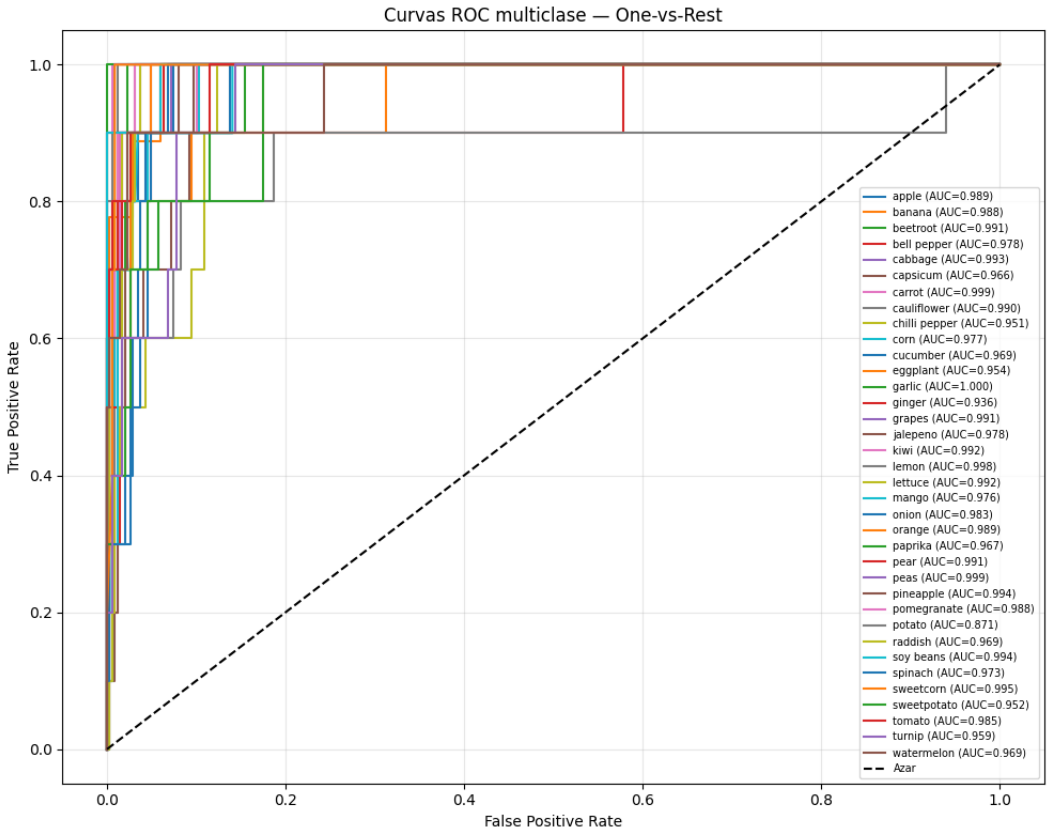


Figura 9: Curvas ROC multiclase (uno contra el resto) de las 36 clases y AUC por clase.

Las 36 clases tuvieron positivos y negativos en el conjunto de prueba y fueron evaluables. El **Macro AUC fue 0.9774**. Los AUC por clase van desde 0.871 (*potato*) hasta 1.000 (*garlic*); *carrot* y *peas* obtienen 0.999 y *lemon* 0.998. Otras clases de menor AUC son *ginger* (0.936), *chilli pepper* (0.951), *sweetpotato* (0.952) y *eggplant* (0.954). Las curvas son escalonadas porque cada clase solo tiene 10 positivos.

Un hallazgo importante es la diferencia entre ranking y decisión: el AUC es alto (0.977) aun cuando la *accuracy* es de 0.624. Un AUC alto indica que el modelo asigna a la clase correcta una probabilidad relativamente mayor que a las demás, aunque no siempre la

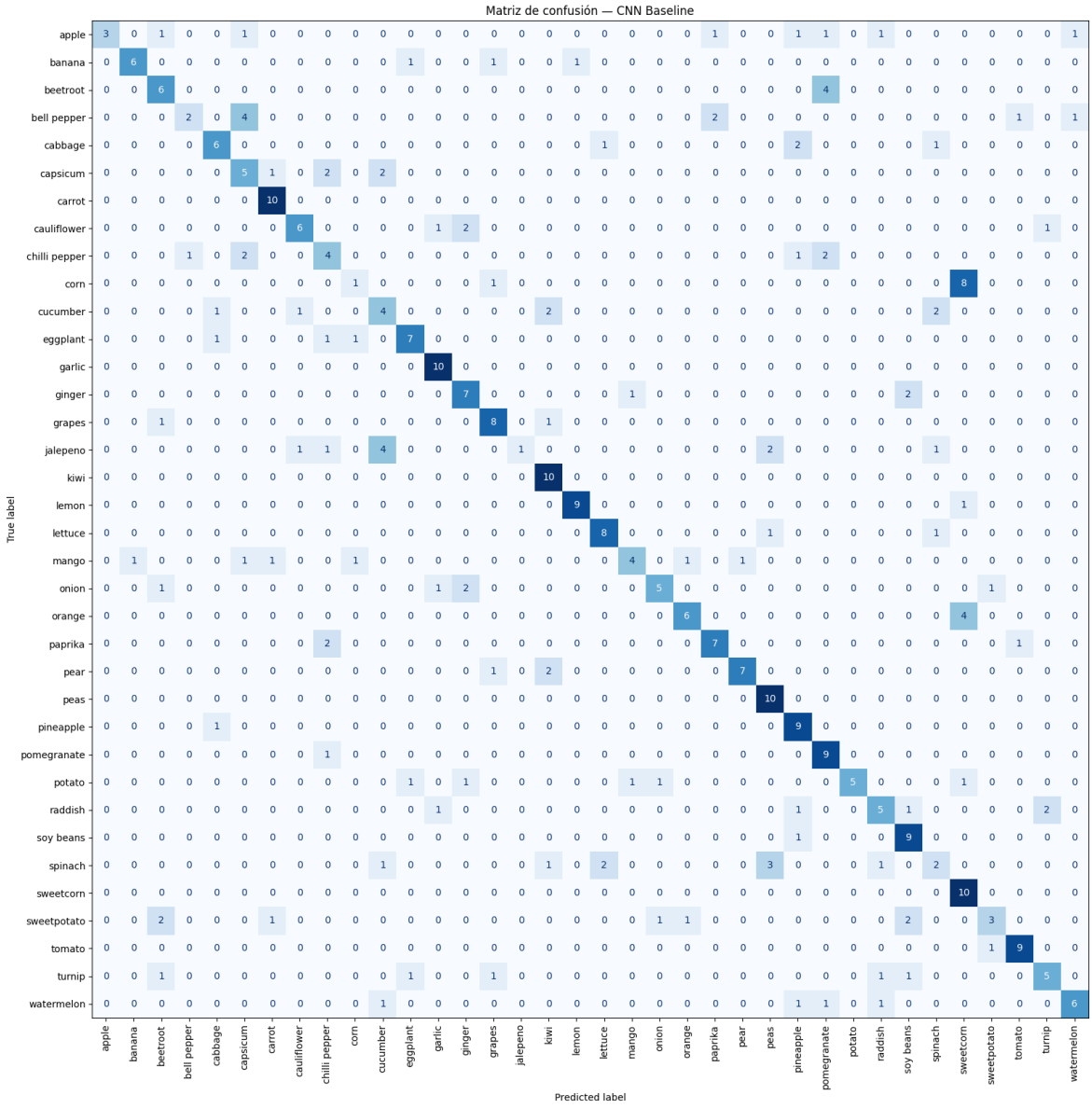


Figura 8: Matriz de confusión de la CNN Baseline en el conjunto de prueba (filas: clase real; columnas: clase predicha).

máxima; esto concuerda con la elevada *accuracy* Top-3. Por tanto, el modelo captura señal discriminativa útil que se pierde cuando se fuerza una sola predicción. El caso de *potato* es ilustrativo: su precisión es perfecta (1.0000) pero su AUC es el más bajo, lo que sugiere que al menos una de sus imágenes recibe una puntuación muy baja y se mezcla con negativos.

4.6. Fase 7: análisis de errores

4.6.1. Pares de clases más confundidos

El modelo cometió 135 errores en el conjunto de prueba. La Tabla 10 lista los diez pares real–predicho más frecuentes, que suman 35 de esos 135 errores (25.9 %).

Tabla 10: Diez confusiones más frecuentes en el conjunto de prueba.

Clase real	Clase predicha	Cantidad
corn	sweetcorn	8
beetroot	pomegranate	4
bell pepper	capsicum	4
jalepeno	cucumber	4
orange	sweetcorn	4
spinach	peas	3
capsicum	chilli pepper	2
bell pepper	paprika	2
chilli pepper	pomegranate	2
capsicum	cucumber	2

4.6.2. Interpretación de las confusiones

- **Ambigüedad de etiquetas (corn → sweetcorn; bell pepper → capsicum/paprika).** Es el error dominante: 8 de las 10 imágenes de *corn* se predicen como *sweetcorn*. En el uso cotidiano, "corn" y "sweetcorn" designan el mismo producto, y lo mismo ocurre con "bell pepper", "capsicum" y "paprika", que en distintas regiones nombran pimientos de la misma especie (*Capsicum annuum*). Es plausible que estas categorías sean semánticamente solapadas más que visualmente distinguibles; si es así, parte del error no es un fallo del modelo sino un problema de definición de clases. Esta es una inferencia razonada a partir de los nombres y de la matriz, que no se comprobó con una inspección imagen por imagen.

- **Similitud de color y forma entre categorías distintas.** *beetroot* → *pomegranate* (4) comparte un rojo oscuro intenso, y *orange* → *sweetcorn* (4) comparte un tono amarillo-anaranjado; en ambos casos el color parece pesar más que la forma, algo esperable en una red pequeña entrenada desde cero.
- **Hortalizas verdes entre sí.** *jalepeno* → *cucumber* (4) y *spinach* → *peas* (3): clases verdes con texturas y fondos similares. *spinach* es la clase con menor F1 entre las no ambiguas por nombre (0.2353).
- **Efecto "sumidero".** *sweetcorn* recibe errores de *corn*, *orange*, *lemon* y otras; esto explica su precisión de solo 0.4167 pese a su recall perfecto.

4.6.3. Inspección visual de errores



Figura 10: Ocho primeras predicciones incorrectas del conjunto de prueba, con su etiqueta real y la predicha.

La Figura 10 debe leerse con cautela: el conjunto de prueba se recorre sin barajar y ordenado por clase, de modo que estos ocho ejemplos corresponden a *apple* (siete) y *banana* (uno), y **no** son una muestra representativa de todos los errores. Aun así, son muy informativos:

- Varias imágenes de *apple* no son fotografías típicas: el logotipo de la marca de tecnología sobre fondo azul (predicha *pineapple*), el logotipo en colores del arcoíris (*capsicum*), y un cartel de "Apple candle" (*pomegranate*). La octava imagen es una caricatura de una banana (predicha *eggplant*). Son etiquetas correctas en términos del texto de búsqueda original, pero visualmente no representan la fruta; ningún clasificador entrenado con fotografías debería acertarlas.

- Otras sí son fotografías claras de manzanas (tres manzanas rojas sobre fondo blanco, un cuenco de manzanas) y fueron mal clasificadas como *beetroot*, *paprika* y *watermelon*. Son fallos genuinos del modelo, posiblemente por el peso del color rojo y la limitada capacidad de la red (hipótesis no comprobada).
- Una imagen de *apple* con franjas negras laterales (predicha *raddish*) muestra el efecto de recortes con relleno, que introducen un patrón ajeno a la clase.

Esto explica en parte el bajo recall de *apple* (0.3000) pese a su precisión perfecta.

4.6.4. Acciones correctivas

Tabla 11: Hallazgos observados y acciones correctivas propuestas.

Hallazgo observado	**Acción correctiva propuesta**
corn/*sweetcorn* (8 errores) y *bell pepper*/*capsicum*/*paprika* (6 errores) son categorías de nombre casi equivalente.	Revisar la taxonomía: fusionar categorías sinónimas o redefinirlas con criterios visuales verificables, y re-etiquetar con revisión manual.
Bajo recall con precisión perfecta en *apple*, *jalepeno* y *potato*.	Aumentar ejemplos de esas clases o aplicar ponderación de clases durante el entrenamiento.
Imágenes no fotográficas (logotipos, caricaturas, carteles) etiquetadas como fruta.	Depurar el conjunto: filtrar logotipos, ilustraciones y capturas con texto; o crear una clase "no fruta".
Confusiones por color: *beetroot*/*pomegranate*, *orange*/*sweetcorn*.	Aumento fotométrico moderado (brillo, saturación) y más ejemplos con fondos y luces variados; considerar mayor resolución o arquitecturas con más capacidad.
Subajuste general (curvas sin converger a las 30 épocas).	Entrenar más épocas con la tasa de aprendizaje ajustada, o usar aprendizaje por transferencia con un modelo preentrenado.

4.7. Fase 8: comparación sistemática de modelos

Tabla 12: Comparación sistemática de los tres modelos.

Modelo	Mejor val. acc.	Menor val. loss	Épocas	Parámetros	Mejor época
CNN Baseline	0.6268	1.3013	30	115 076	26
CNN Mejorada	0.6239	1.4016	30	329 860	30
CNN Optimizada	0.5100	1.7353	30	335 812	29

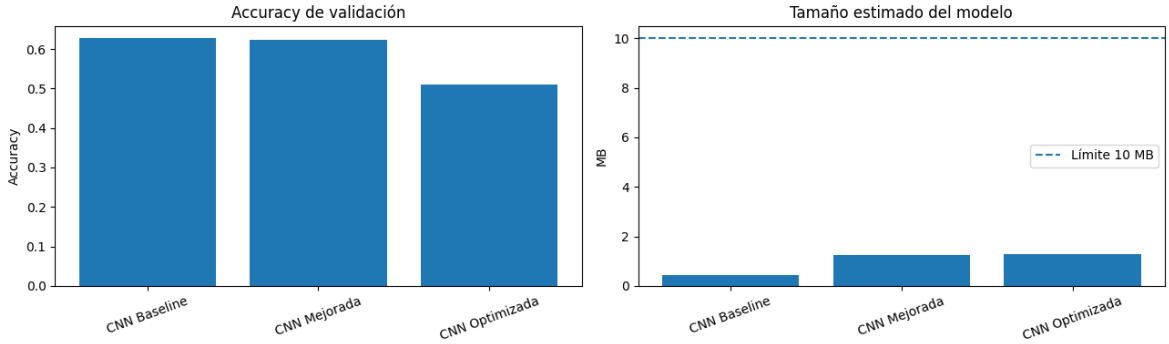


Figura 11: Izquierda: *accuracy* de validación por modelo. Derecha: tamaño estimado en FP32 frente al límite de 10 MB.

Interpretación. Más parámetros no implicaron mejor rendimiento: la CNN Mejorada, con 2.87 veces los parámetros de la base, no la supera en validación (0.6239 frente a 0.6268) y tiene mayor pérdida. La CNN Optimizada, la más compleja y regularizada, es la peor (0.5100). Una explicación coherente con las curvas, pero **no verificada con experimentos de ablación**, es que el modelo ya estaba subajustado y las regularizaciones adicionales (L2, un segundo *dropout*, aumento duplicado) ralentizaron aún más el aprendizaje dentro del presupuesto fijo de 30 épocas. Los tres modelos cumplen el límite de 10 MB con holgura (Figura 11, derecha); el criterio de producción favorece entonces a la CNN Baseline, que además es la más pequeña.

Límites de la comparación. Cada modelo se entrenó una sola vez con una semilla, y el conjunto de validación solo tiene 351 imágenes; diferencias de uno o dos puntos porcentuales no son concluyentes.

4.8. Fases 9, 10 y 11: versionado, *logging* y documentación

Fase 9 – Versionado. El modelo seleccionado se guardó como `comidas_verduras_best.keras` junto con un archivo `metadata.json` que registra la información necesaria para reproducir y auditar el experimento (Tabla 13).

Tabla 13: Metadatos del modelo en producción (`metadata.json`).

Campo	**Valor**
Versión	1.0
Fecha de creación	2026-11-11, 03:39
Modelo seleccionado	CNN Baseline (`cnn_baseline`)
Forma de entrada	$224 \times 224 \times 3$
Número de clases	36
Parámetros	115 076
Tamaño estimado FP32	0.4390 MB
Test accuracy	0.6240
Test loss	1.3205
Macro AUC	0.9774
TensorFlow / Python	2.21.0 / 3.13.16

Para verificar la integridad del archivo guardado, se recargó el modelo y se reevaluó en el conjunto de prueba, obteniendo nuevamente una *accuracy* de 0.6240, idéntica a la original. Esto confirma que el guardado y la carga no alteran los pesos.

Fase 10 – *Logging.* Cada experimento generó un directorio propio con su configuración (`config.json`), los *checkpoints*, un registro CSV por época (`CSVLogger`) y registros de *TensorBoard*. Los directorios se identifican con el nombre del modelo y la marca de tiempo (por ejemplo, `cnn_baseline_20261111_013231`), lo que permite trazar qué ejecución produjo cada resultado.

Fase 11 – Documentación. El presente documento constituye la documentación técnica del proyecto; el *notebook* incluye además una lista de verificación de entrega de las doce fases.

4.9. Fase 12: despliegue y conversión a TensorFlow Lite

El modelo se convirtió a TensorFlow Lite en dos variantes: sin cuantizar (float32) y con cuantización a float16 [7]. La conversión confirmó una entrada de forma `(None, 224, 224, 3)` y una salida de forma `(None, 36)`.

Tabla 14: Tamaño real de los archivos TensorFlow Lite.

Variante	**Tamaño real (MB)**	**¿Cumple < 10 MB?**
TFLite float32	0.44	Sí
TFLite float16 (cuantizado)	0.22	Sí

Interpretación. El tamaño real del archivo float32 (0.44 MB) coincide con la estimación teórica (0.4390 MB), y la cuantización a float16 lo reduce a la mitad (0.22 MB), como se espera al almacenar cada peso con 2 bytes en lugar de 4. Ambas variantes cumplen con amplitud el límite de 10 MB. Se implementó además una función `predict_image` que carga el modelo, normaliza una imagen externa y muestra las tres clases más probables; sin embargo, no se ejecutó con fotografías externas, y tampoco se midió la *accuracy* del modelo float16 ni su latencia en un dispositivo real, por lo que el despliegue verificado es de tamaño y de conversión, no de desempeño en el dispositivo.

5. Discusión

Desempeño global. Una *accuracy* de 62.4 % en 36 clases con un modelo de solo 115 mil parámetros entrenado desde cero es un resultado sólido frente a las líneas base (3.14 % y 2.78 %), pero insuficiente para un producto final. El Top-3 de 86.1 % y el Macro AUC de 0.977 muestran que el modelo ha aprendido una representación útil; lo que falla es la decisión final entre clases cercanas.

Subajuste frente a sobreajuste. Las curvas indican que la limitación principal es de optimización y capacidad, no de generalización: la brecha entre entrenamiento y validación es negativa (validación mejor) y las pérdidas seguían descendiendo en la época 30. Una consecuencia práctica es que `EarlyStopping` nunca actuó, y que el presupuesto de 30 épocas probablemente es demasiado corto para estas arquitecturas.

La arquitectura más compleja no fue la mejor. Este resultado contradice la expectativa de que las conexiones residuales y la regularización mejoran siempre el rendimiento. Con un conjunto pequeño (3 115 imágenes de entrenamiento), pocas épocas y subajuste, la regularización adicional no es beneficiosa. Además, el aumento doble de la CNN Optimizada pudo agravar el problema. Estas explicaciones son hipótesis coherentes con la evidencia, no conclusiones demostradas.

La calidad de los datos como techo. El análisis de errores sugiere que una parte del error es atribuible al conjunto de datos: categorías sinónimas y logotipos o ilustraciones etiquetados como frutas. Mejorar los datos probablemente rinda más que seguir ajustando arquitecturas.

Validez de los resultados. La prueba tiene solo 10 imágenes por clase, por lo que cada imagen mal clasificada mueve el recall de una clase en 10 puntos porcentuales y las métricas por clase son muy inestables. Además, cada modelo se entrenó una sola vez. Las conclusiones por clase deben tomarse como indicativas.

6. Conclusiones

1. Se construyó un flujo completo y reproducible de doce fases para clasificar 36 categorías de frutas y hortalizas a partir de 3 825 imágenes, sin imágenes corruptas y con un desbalance leve (razón 1.36 entre la clase mayor y la menor).
2. Se compararon tres CNN con idéntica configuración. La CNN Baseline (115 076 parámetros) obtuvo la mejor *accuracy* de validación (0.6268) y la menor pérdida (1.3013); la Mejorada quedó prácticamente empatada (0.6239) y la Optimizada fue la peor (0.5100).
3. En prueba, el modelo seleccionado alcanzó *accuracy* 0.6240, Top-3 0.8607, Macro F1 0.6025 y Macro AUC 0.9774, superando unas 20 veces la línea base mayoritaria. El resultado de prueba es consistente con el de validación.
4. Las curvas de aprendizaje muestran subajuste y no sobreajuste; ningún modelo convergió en 30 épocas.
5. Los errores se concentran en pares de clases de nombre o apariencia casi equivalentes (*corn*/*sweetcorn*, *bell pepper*/*capsicum*/*paprika*) y en imágenes no fotográficas, lo que apunta a la calidad y definición de los datos como principal oportunidad de mejora.
6. El modelo se exportó a TensorFlow Lite con 0.44 MB (0.22 MB en float16), cumpliendo con holgura el límite de 10 MB.

6.1. Limitaciones

Cada modelo se entrenó una sola vez; el conjunto de validación (351) y el de prueba (359) son pequeños; no se realizaron ablaciones para confirmar las causas del menor rendimiento de la CNN Optimizada; el suavizado de etiquetas declarado en la configuración no se aplicó; no se evaluó el modelo con fotografías externas ni la variante float16 en desempeño o latencia.

6.2. Trabajo futuro

- Entrenar más épocas con programación de la tasa de aprendizaje y verificar la convergencia.
- Aplicar aprendizaje por transferencia con redes preentrenadas ligeras, como MobileNet [5], manteniendo el límite de 10 MB.
- Depurar el conjunto de datos: fusionar o redefinir clases sinónimas y eliminar

imágenes no fotográficas.

- Repetir los entrenamientos con varias semillas y validación cruzada para estimar la variabilidad.
- Evaluar el modelo con fotografías tomadas fuera del conjunto de datos y medir la latencia de la versión TFLite en un teléfono real.

Referencias

[1] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 265–283.

[2] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.

[3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

[5] Howard, A. G., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.

[6] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 448–456.

[7] Jacob, B., et al. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704–2713.

[8] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

[9] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.

[10] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*.

[11] Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), 60.

[12] Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437.

[13] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958.

A. Código principal del proyecto

Se incluyen los fragmentos esenciales del *notebook* (sin tildes en los comentarios para compatibilidad con el paquete `listings`).

A.1. Pipeline de datos (Fase 3)

```
normalization = tf.keras.layers.Rescaling(1./255, name='normalization')
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip('horizontal'),
    tf.keras.layers.RandomRotation(0.08),
    tf.keras.layers.RandomZoom(0.10),
], name='data_augmentation')

def prepare_train(images, labels):
    images = normalization(images)
    images = data_augmentation(images, training=True)
    return images, labels

def prepare_eval(images, labels):
    return normalization(images), labels
```

A.2. CNN Baseline (Fase 4)

```
def conv_block(x, filters, pool=True):
    x = tf.keras.layers.Conv2D(filters, 3, padding='same', use_bias=False)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    if pool:
        x = tf.keras.layers.MaxPooling2D()(x)
    return x

def build_baseline(input_shape, num_classes):
    inputs = tf.keras.Input(shape=input_shape)
    x = conv_block(inputs, 32)
    x = conv_block(x, 64)
    x = conv_block(x, 128)
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dense(128, activation='relu')(x)
    x = tf.keras.layers.Dropout(0.4)(x)
    outputs = tf.keras.layers.Dense(num_classes, activation='softmax')(x)
    return tf.keras.Model(inputs, outputs, name='cnn_baseline')
```

A.3. Bloque residual (Fase 4)

```
def residual_block(x, filters, stride=1):
    shortcut = x
    y = tf.keras.layers.Conv2D(filters, 3, strides=stride, padding='same', use_bias=
        False)(x)
    y = tf.keras.layers.BatchNormalization()(y)
    y = tf.keras.layers.Activation('relu')(y)
    y = tf.keras.layers.Conv2D(filters, 3, padding='same', use_bias=False)(y)
    y = tf.keras.layers.BatchNormalization()(y)
    if stride != 1 or int(shortcut.shape[-1]) != filters:
        shortcut = tf.keras.layers.Conv2D(filters, 1, strides=stride, padding='same',
            use_bias=False)(shortcut)
        shortcut = tf.keras.layers.BatchNormalization()(shortcut)
    y = tf.keras.layers.Add()([y, shortcut])
    return tf.keras.layers.Activation('relu')(y)
```

A.4. Entrenamiento con cinco callbacks (Fase 5)

```
optimizer = tf.keras.optimizers.AdamW(learning_rate=1e-3, weight_decay=1e-4)
model.compile(
    optimizer=optimizer,
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=['accuracy',
             tf.keras.metrics.SparseTopKCategoricalAccuracy(k=3, name='top_k_accuracy
                 ')])

callbacks = [
    tf.keras.callbacks.ModelCheckpoint(checkpoint_path, monitor='val_accuracy',
                                       mode='max', save_best_only=True),
    tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=7,
                                     restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                                         patience=3, min_lr=1e-6),
    tf.keras.callbacks.TensorBoard(log_dir=log_dir),
    tf.keras.callbacks.CSVLogger(csv_path)]

history = model.fit(train_data, validation_data=validation_data,
                    epochs=30, callbacks=callbacks)
```

A.5. Conversión a TensorFlow Lite (Fase 12)

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT] # solo variante cuantizada
```

```
converter.target_spec.supported_types = [tf.float16]
tflite_bytes = converter.convert()
```